

Examen Parcial de IA

(4 de noviembre de 2024)

Duración: 120 min

1. (6 puntos) Una empresa del mundo del espectáculo gestiona los contratos de N grupos musicales que se dedican a actuar en las fiestas de los pueblos. Cada grupo musical tiene una tarifa por actuación, de la que una parte se la queda el grupo y otra la empresa. Por otro lado, los municipios celebran sus fiestas patronales en unas ciertas fechas y dedican una parte de su presupuesto festivo, X euros, a pagar las actuaciones de los grupos musicales. El problema es organizar las giras de los grupos de modo que se cubran las peticiones de los pueblos, en total P actuaciones (puede haber más de una actuación en un pueblo en días consecutivos), sin pasarse del presupuesto municipal, se maximice la recaudación de la empresa y se minimicen los kilómetros que al cabo de la temporada ha de realizar cada grupo. Además, el mismo grupo no puede actuar más de un día en el mismo pueblo y cuando acaban en un pueblo, se van al siguiente en el que tienen actuación.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística, ...). El objetivo es comentar la solución que se propone respecto a si es correcta, es eficiente, o es mejor o peor respecto a otras alternativas posibles. Justifica tu respuesta.

- a) Se propone usar Hill-Climbing. La solución de partida coloca, en orden creciente de fecha, un grupo diferente en cada una de las fechas previstas de actuación y cada grupo se asigna una sola vez. El operador disponible es `poner_grupo` y la función heurística es:

$$h'(n) = \frac{\sum_{\forall grupo_i} Kilometros_recorridos(grupo_i)}{\sum_{\forall grupo_i} Dinero_recaudado_empresa(grupo_i)}$$

(entre paréntesis: puntuación de cada parte, puntuación máxima total es 2,0)

(0,2) A priori Hill Climbing es una técnica adecuada para intentar encontrar soluciones que maximicen o minimicen criterios si se tiene un heurístico que pueda guiar bien la exploración de los operadores.

(0,2) La solución inicial propuesta ordena las fechas de las actuaciones y luego asigna grupos, teniendo un coste de generación de $O(P \log P + N)$ (ordenar las P fechas y luego asignar los N grupos una vez).

(0,1) Sin embargo esta ordenación de fechas no aporta nada a la calidad de la solución inicial (sería más útil ordenar los grupos por su rendimiento económico, o actuaciones por distancia en km a un punto de partida -si hubiera un único punto de partida para todos los grupos, que no es el caso-).

(0,2) La solución inicial puede ser no-solución, ya que no se controla el presupuesto del municipio a la hora de asignar grupo (pudiendo superarlo y, por lo tanto, obtener una solución inicial fuera del espacio de soluciones), y también podemos tener el caso en el que el número de grupos es menor que el de actuaciones, y la generación de la solución inicial propuesta deja actuaciones sin cubrir (y por lo tanto, genera de nuevo una no-solución).

(0,2) El operador `poner_grupo` podría completar la solución parcial generada cuando hay menos grupos que actuaciones y meternos en el espacio de soluciones, pero su gran problema es que no comprueba que las asignaciones no superen el presupuesto de los pueblos, ni la asignación consecutiva de grupos en las actuaciones de un mismo pueblo, por lo que puede generar nuevas no-soluciones.

(0,2) Su factor de ramificación es $O(N \times P)$ (colocar cualquiera de los grupos en cualquiera de las actuaciones por asignar).

(0,2) Solo con el operador `poner_grupo` no podremos cubrir todo el espacio de soluciones ya que las asignaciones del estado inicial no se podrán cambiar.

(0,1) Para maximizar la recaudación de la empresa y minimizar los km recorridos, el heurístico propuesto se debería minimizar.

Sin embargo hay varios problemas con este heurístico.

(0,1) El primero es que, como hay más de un criterio en la fórmula, se debería añadir pesos al menos a alguno de los criterios para poder ajustar mejor la influencia de cada uno de ellos en el heurístico.

(0,2) El segundo problema viene de la elección de la división para combinar los criterios, ya que suele ser difícil de ajustar, dando el heurístico valores similares para soluciones con muchos km y dinero recaudado, y también con pocos km y poco dinero recaudado.

(0,1) De hecho en este caso el mínimo de la función se consigue recorriendo 0 km, por lo que el heurístico, al minimizarlo, tenderá a la solución vacía, y por ello, partiendo de una solución inicial, hará que no se llegue a aplicar el operador **poner_grupo** excepto en casos extremos donde la nueva actuación requiere muy pocos km de recorrido y se obtienen grandes beneficios para la empresa.

(0,2) Y el tercer problema (quizás el más importante) es que, dado que el algoritmo se puede encontrar con no-soluciones, el heurístico debería penalizarlas para forzar la aplicación de operadores que lleven de nuevo al espacio de soluciones.

- b) Se propone usar Hill-Climbing. La solución de partida se calcula de la siguiente manera: se ordenan los grupos en orden decreciente de beneficio para la empresa y se empiezan a asignar siguiendo ese orden a cada una de las fechas disponibles. Si cuando ya se han colocado todos los grupos una vez todavía quedan fechas sin cubrir, se repite el proceso las veces que sea necesario. Se respeta que el mismo grupo no actúe más de una vez en el mismo pueblo. El operador es **intercambiar_grupo** siempre y cuando no suponga que se supera el presupuesto del municipio y se respete una única actuación por grupo en el mismo pueblo. La función heurística es:

$$h'(n) = \sum_{\forall grupo_i} Kilometros_recorridos(grupo_i) - \sum_{\forall grupo_i} Dinero_recaudado_empresa(grupo_i) * 100$$

(entre paréntesis: puntuación de cada parte, puntuación máxima total es 2,0)

(0,1) De nuevo plantear el uso de Hill Climbing en este problema, que necesita maximizar/minimizar criterios, es adecuado a priori, siempre que tengamos un heurístico que guíe bien la exploración de los operadores.

En este caso la forma de obtener la solución inicial es un generador que da una asignación completa de grupos a actuaciones, asignando primero los grupos más rentables para la empresa.

(0,2) El coste de generación en este caso es $O(N \log N + P)$ (ordenar los N grupos y luego asignarlos secuencialmente a las P fechas de actuación), balanceando el número de actuaciones por grupo (como mucho algunos grupos más rentables tendrán una actuación más que los menos rentables).

(0,2) A pesar de este coste de generación la calidad puede ser entre media y mala, ya que en el mejor de los casos es solución y aun tiene margen de mejora (asignarle más actuaciones a los grupos más rentables).

(0,3) Pero en el peor de los casos es una no-solución, ya que el generador de la solución inicial no tiene en cuenta los presupuestos de los municipios, por lo que podría no ser una solución válida.

(0,2) El operador **intercambiar_grupo** entre grupos ya asignados que se propone, al controlar las restricciones, no solo no saca al algoritmo del espacio de soluciones, sino que incluso podría ir arreglando algunas asignaciones de grupos a actuaciones fuera del presupuesto del municipio (si el heurístico penalizara las no-soluciones, cosa que no hace).

(0,2) Pero este operador, al no poder incluir ningún grupo nuevo en la solución, no nos permite cubrir todo el espacio de soluciones y, además, hace que el beneficio de la empresa durante la exploración del Hill Climbing sea siempre un valor constante.

(0,1) El factor de ramificación es $O(P \times P)$.

(0,1) Una solución algo mejor será un operador `cambiar_grupo`, que simplemente cambia el grupo asignado a una actuación cumpliendo las restricciones presupuestarias y de fechas consecutivas. Este operador tiene un factor de ramificación asintóticamente similar (es $O(N \times P)$) y permite tanto el aumentar actuaciones de los grupos más rentables como añadir grupos a la solución que quedaron fuera en la solución inicial.

(0,2) Esta función la tendríamos que minimizar para obtener soluciones de menos kilómetros y mayor recaudación. Aunque es algo poco intuitivo tener que minimizar una función que dará siempre un valor negativo, sería más intuitivo girar la formula y maximizarla:

$$h''(n) = 100 * \sum_{\forall grupo_i} Dinero_recaudado_empresa(grupo_i) - \sum_{\forall grupo_i} Kilometros_recorridos(grupo_i)$$

(0,1) Tanto en el heurístico propuesto como en su versión complementaria, al ponderar la recaudación pesará más este criterio en las soluciones. En esta función una ponderación es una buena idea no solo para darle más peso a un criterio sobre otro, podría ser necesaria por que las unidades en que se mide cada criterio pueden suponer un desequilibrio entre los criterios, por lo que solo faltaría ver de forma experimental si 100 es la ponderación correcta.

(0,1) Además, con el operador propuesto, el sumatorio del beneficio dará siempre el mismo valor, por lo que el heurístico solo guiará la exploración a reducir los kilómetros recorridos y a reducir las no soluciones (si las penalizáramos).

(0,2) Pero de nuevo tiene el mismo problema importante del heurístico en el apartado anterior: no penaliza las no-soluciones respecto a las soluciones.

- c) Se plantea resolverlo con algoritmos genéticos. Se asigna un identificador numérico a cada uno de los grupos que se codifica en binario como una cadena de b bits, la solución se representa como una tira de $P \times b$ bits. La solución de partida se calcula de la siguiente forma: se ordenan los grupos en orden decreciente de beneficio para la empresa, se toma el primer grupo y se le asigna la primera fecha libre sin pasarse del presupuesto del municipio, luego se busca la siguiente fecha libre en la que pueda actuar el mismo grupo, y tal que suponga un desplazamiento mínimo para el grupo y que no supere el presupuesto del municipio, se le asigna el grupo y se repite el proceso hasta que no queden más fechas posibles para ese primer grupo. Se repite exactamente el mismo proceso para los grupos sucesivos hasta que ya no queden fechas por cubrir o no haya más grupos.

Como operadores genéticos se utiliza solamente el operador de cruce, donde los puntos de cruce no pueden estar dentro de la identificación de un grupo. El heurístico es el siguiente:

$$h'(n) = \sum_{\forall grupo_i} Kilometros_recorridos(grupo_i) \times \frac{Dinero_cobrado(grupo_i)}{Dinero_recaudado_empresa(grupo_i)}$$

(entre paréntesis: puntuación de cada parte, puntuación máxima total es 2,0)

(0,1) A priori el uso de algoritmos genéticos podría ser adecuado para este problema, donde buscamos minimizar y maximizar criterios sin requerir el óptimo. Aunque con heurísticos adecuados un Hill Climbing podría funcionar mejor y ser más eficiente que un algoritmo genético.

(0,2) La representación de la solución es correcta, ya que podemos representar todas las asignaciones de grupos a actuaciones.

(0,2) Pero también se pueden representar muchas no-soluciones. Si el número de grupos N no es una potencia de 2, la codificación propuesta podría también representar identificadores de grupo que no existen, pero en este caso no es un problema ya que la solución inicial genera siempre identificadores correctos que no son modificados por el operador de cruce. La codificación también permite representar otros tipos de no-solución: exceder el presupuesto de un municipio, asignaciones de un grupo a dos actuaciones en la misma fecha, o la asignación de un grupo a actuaciones seguidas en el mismo municipio.

(0,1) El tamaño de la representación escogida es $O(P \log_2 N)$ bits, que es la forma más pequeña en espacio para poder representar asignaciones de grupos a actuaciones.

La solución inicial es un generador con una estrategia avariciosa que asigna grupos a actuaciones, asignando primero los grupos más rentables para la empresa y seleccionando la siguiente actuación más cercana.

(0,1) Este método de generar la solución inicial es bastante costoso ($O(N \log N + N \times P)$).

(0,1) Generará una asignación válida siempre que haya grupos suficientes para cubrir las actuaciones que quedan con el presupuesto restante de los municipios, en caso contrario podemos tener no-soluciones. Por ejemplo, si asignamos los grupos que dan más beneficio es probable que sean los más caros, gastando muy rápido el presupuesto de los municipios con un número menor de grupos, de forma que no haya forma de cubrir el resto de las actuaciones que faltan (es decir, habría casos en los que el generador propuesto no consigue generar una solución completa, para evitarlo habría que modificarlo, haciendo backtracking no cronológico eliminando asignaciones anteriores). La calidad de las soluciones (si lo son) será relativamente buena en lo que respecta al beneficio ya que se asignan primero los que reportan mayor beneficio. Respecto a las distancias la asignación utiliza un algoritmo avaricioso para minimizar la distancia, lo que también podría dar soluciones bastante buenas, que colocan el algoritmo ya cerca de los óptimos locales (algo que sería problemático para Hill Climbing, pero no lo es para algoritmos genéticos con operadores y funciones de evaluación adecuadas).

(0,2) De todas formas, la forma de obtener la solución inicial no es correcta para solucionar el problema mediante algoritmos genéticos porque el generador de soluciones iniciales propuesto siempre genera la misma solución inicial y en este caso necesitamos obtener una población de soluciones iniciales diferentes. Una forma de generar una solución inicial de calidad similar y coste de generación similar sería escoger de forma aleatoria la "siguiente fecha libre".

(0,2) En este caso se ha optado por un operador de cruce que no puede cruzar dentro de la codificación de los grupos, lo cual nos asegura que los identificadores de grupo sigan siendo válidos en la generación siguiente si lo eran en la generación anterior. Pero aun así este operador puede generar no-soluciones (por ejemplo, si al cruzar dos soluciones padre genera en uno de los hijos una secuencia de codificaciones donde el mismo grupo actúa dos veces seguidas en actuaciones del mismo municipio). Y esto no es controlado ni penalizado por el heurístico que se usa para evaluar las soluciones.

(0,1) El segundo problema del operador de cruce propuesto, unido a la falta de operador de mutación es que las soluciones generadas tras el cruce solo serán variaciones de las soluciones iniciales, por lo que el operador solo puede acceder a soluciones vecinas de las soluciones de la generación anterior y no tenemos por lo tanto una cobertura de todo el espacio de soluciones.

(0,1) Si además le añadimos que el generador de soluciones iniciales nos dará una población donde todas las soluciones son iguales, el resultado tras cada cruce serán dos soluciones hijas exactamente iguales a sus padres, por lo que el algoritmo no podrá avanzar por el espacio de soluciones.

(0,2) Para poder minimizar kilómetros recorridos deberíamos minimizar la función h' que nos proponen, Pero los algoritmos genéticos consideran la función de evaluación como un

afunción de *fitness* que siempre se ha de maximizar, por lo que deberíamos girar la función haciendo que los kilómetros resten.

(0,1) En la función heurística se ha añadido un criterio más, el dinero cobrado por los grupos, que no es un criterio que el enunciado nos pida minimizar.

(0,1) En el heurístico propuesto (sin girar) estaremos minimizando el beneficio relativo entre lo que se queda el grupo y lo que se queda la empresa (que en principio, para cada grupo en la solución, es un valor constante). Por ello minimizar la h' propuesta es equivalente a minimizar únicamente los kilómetros, y eso vuelve a presionar hacia soluciones en que no se hagan actuaciones.

(0,2) Y como ya se ha dicho, el heurístico debería penalizar las posibles no-soluciones (exceder el presupuesto de municipios, grupos que realizan dos actuaciones seguidas en el mismo municipio, grupos que actúan en dos municipios el mismo día).

2. (4 puntos) En contextos de guerra, muchas veces es necesario rescatar a grupos de supervivientes dentro de regiones amplias, no conectadas, y de bajos recursos. Para ello, se organizan convoyes de transportes que viajen por la región, rescatando a estas personas, consiguiendo recursos de forma autosuficiente (como gasolina, comida, etc). Disponemos de T transportes de evacuación independientes, y R grupos a rescatar (en sitios). El objetivo es minimizar el tiempo que tardamos en rescatar al último grupo. Los transportes viajan de sitio a sitio, tardando $\text{tiempo}(s1, s2)$ (**asumiendo concurrencia en desplazamientos de varios vehículos**), y consumiendo una autonomía $\text{desgaste}(s1, s2)$. Cada transporte tiene su tiempo acumulado $\text{tiempo}(tr)$ y su propia autonomía $\text{autonomia}(tr)$ (que nunca puede quedar < 0). Además de sitios con personas a rescatar, hay caches de recursos, que recuperan la autonomía en $\text{recursos}(c)$. Al visitar una cache, un transporte recoge todos sus recursos, y la cache queda vacía.

Damos por terminado el problema cuando todos los grupos de rescate han sido rescatados, y el tiempo de rescate como el tiempo máximo que tarda un transporte en hacer su ruta completa.

- a) Queremos utilizar A^* . Empezamos con una solución vacía, con autonomías y sitios actuales de cada transporte especificados por el problema y tiempos de ruta de cada camión a 0. Como operadores usamos dos. El primero es $\text{rescatar}(tr, s)$, que tiene como efecto que el transporte tr pasa a estar en sitio s (de rescate), el tiempo del transporte tr incrementa acorde con el desplazamiento ($\text{tiempo}(\text{sitio_actual}(tr), s)$). Condición de aplicabilidad: $\text{autonomia}(tr) \geq \text{desgaste}(\text{sitio_actual}(tr), s)$. El coste es 'cuánto tiempo se pasa el desplazamiento de ese transporte del tiempo máximo actual': formalmente $\max(0, \text{tiempo}(tr) + \text{tiempo}(\text{sitio_actual}(tr), s) - \max_i(\text{tiempo}(i)))$. Por ejemplo, si al principio movemos un camión con tiempo 2, y luego otro camión con tiempo 3, los costes de los operadores serán 2 y 1 respectivamente. Aparte, tenemos operador $\text{repostar}(tr, c)$, que reposta en una cache. Es lo mismo que el operador rescatar en todo, salvo que en su efecto incrementa también la autonomía en $\text{recursos}(c)$. La función heurística es la suma de, para cada transporte, el mínimo desgaste a un grupo de personas pendiente de ser rescatadas.

El objetivo didáctico de este problema era presentar un caso de A^* donde el coste algorítmico no es siempre positivo (debido a la concurrencia), pero con la misma problemática en el heurístico que en la mayoría de problemas que hemos visto con A^* .

- (0.2) En la versión más completa de una respuesta, a priori A^* no tiene buena pinta ya que es difícil pensar un heurístico admisible que estime cuánto tiempo falta hasta rescatar a todo grupo. En caso de que exista, puede ser buena idea. El problema no indica si se debe buscar el tiempo mínimo posible, pero asumiríamos que sí dado que se trata de vidas humanas en un contexto de guerra. Cualquier justificación sobre que sea bueno por obtener un óptimo global, que no lo sea por no requerirse y ser un algoritmo ineficiente, o que involucre la dificultad de obtener un heurístico admisible se ha dado por válida. **Como advertimos, se ha penalizado extra si la justificación dice que este problema es bueno para A^* por tratarse de encontrar un camino.**
- (0.2) Los operadores partiendo de la solución inicial exploran todo el espacio de soluciones.
- (0.2) La ramificación de los operadores es de $T * R$ y $T * C$ respectivamente
- (0.3) Los operadores no actualizan la autonomía de los transportes al moverlos, debería restarse.
- (0.2) El operador repostar permite visitar la misma cache múltiples veces, falta condición de aplicabilidad. No sería mala idea añadir la misma condición al de rescate.
- (0.2) El coste, si bien es raro, es lo que se pide optimizar.
- (0.2) ¡El coste de un operador puede ser 0 cuando hay un transporte por delante! A^* parece que podría quedarse por tanto atascado. Esto no es el caso, ya que en un número finito de desplazamientos otro transporte requerirá pasarse del tiempo. También nos indica que este algoritmo puede ser algo naive en la exploración (se abrirán en paralelo muchos operadores de coste 0).

- (0.2) La heurística está en unidades de desgaste, mientras que el coste está en tiempo. El problema pide tiempo. En ningún caso deberíamos añadir un ponderador con desgaste, en A^* eso no sirve.
- **(0.5) La heurística es claramente no admisible.** Con un grupo restante y cinco transportes, contaríamos el coste de desplazar 5 transportes en el heurístico, mientras que con desplazar uno (el más cercano) es suficiente.

Puntuación máxima total: 2.2, con puntuación adicional en detalles extra como analizar el tamaño del espacio de estados. Como curiosidad, negligiendo caches, debería ser del orden de $R! + (R-1)! + (R-2)! + \dots$ con un factor máximo de $(R/T)!^T$ aproximadamente. El primer término es toda permutación de grupos rescatada por un transporte, el segundo es todos menos uno en un transporte y el restante en otro... hasta tener toda posible permutación de aproximadamente R/T grupos de rescate a rescatar por cada camión.

- b) Se plantea usar un algoritmo de optimización de restricciones. Como variables tenemos diversos grupos. Variables visita ($V_{tr,k}$) para cada transporte y posición k de la ruta, asumiendo que para cada transporte tenemos un máximo de $K = \lceil R/T \rceil$ sitios a visitar ($T \times K$ variables), con dominios todos los R sitios de rescate, C caches y vacío (que indica que el transporte no se mueve). Para cada transporte, tenemos K variables de autonomía ($A_{tr,k}$), con dominios numéricos de 0 a un número muy alto, y K variables de tiempo ($TI_{tr,k}$) con dominio similar. Para las restricciones, asumimos que $\text{desgaste}(s, \text{vacío}) = \text{tiempo}(s, \text{vacío}) = 0$, y $\text{desgaste}(\text{vacío}, s) = \text{tiempo}(\text{vacío}, s) = +\infty$ (impedirá salirnos de vacío si hemos entrado), y para incluir los recursos de las caches lo añadimos al desgaste: $\text{desgaste_con_recursos}(s1, s2) = \text{desgaste}(s, s2) - \text{recursos}(s2)$. Restricciones tenemos: Para cada transporte, una restricción unaria para las condiciones iniciales del problema: $A_{tr,0} = TI_{tr,0} = 0$, y $V_{tr,0}$ el origen del transporte. Restricciones cuaternarias entre cada par de variables de ruta consecutivas de un mismo transporte ($V_{tr,k}, V_{tr,k+1}$) y sus correspondiente de autonomía ($A_{tr,k}, A_{tr,k+1}$) tal que $A_{tr,k+1} = A_{tr,k} - \text{desgaste_con_recursos}(V_{tr,k}, V_{tr,k+1})$. Lo mismo con el tiempo. Finalmente, metemos una variable objetivo a minimizar, con una restricción: $obj = \max_{tr}(V_{tr,K}, V_{tr,K})$.

El objetivo didáctico de este problema era plantear un problema de restricciones que parece complejo por tener muchas variables, pero que requiere de pocas restricciones y de cardinalidad baja. El problema tiene muchos errores fáciles de resolver sin mayor modificación.

- (0.2) Constraint programming no parece malo para el problema ya que es fácil de representar con variables, y podemos optimizar criterios. **Como advertimos se ha penalizado decir que CP no permite optimizar.**
- (0.3) Las variables y sus dominios son, a priori correctas y permiten identificar soluciones únicas. **Se ha penalizado decir que la variable visita no es necesaria.**
- (0.2) Sin embargo, la K escogida parece muy pequeña, ya que fuerza a cada camión a rescatar un número máximo de grupos sin apenas tener margen para repostar. $K = R + C$ es la K mínima para asegurar cualquier posible solución, pero algo menor podría ser válido también en un problema promedio.
- (0.2) Las constraints presentadas son mayormente correctas.
- (0.2) La autonomía al inicio ($A_{tr,0}$) debería ser la autonomía del transporte y no 0.
- (0.3) El truco usado para subsumir el repostar dentro del desgaste es incorrecto, ya que no comprueba bien la autonomía cuando se va a un centro de recursos. Por ejemplo, un transporte con autonomía 1 puede hacer un viaje con desgaste 4 a una caché de repostaje, siempre y cuándo la caché tenga recursos >3 . Debería añadirse una comprobación adicional (restricciones trinarias).
- **(0.4) No se comprueba que todos los grupos queden rescatados.** De hecho, la asignación óptima para este problema es que ningún transporte se mueva en absoluto.

Se requiere de una restricción n-aria (y al haber la posibilidad de 'vacío', no se puede descomponer en restricciones binarias).

- (0.2) No se comprueba que una caché no sea visitada 2 veces.
- (0.2) Para ayudar a la eficiencia, puede ser buena idea impedir revisitas a grupos de rescate también.
- (0.2) La variable objetivo es correcta salvo que mira el 'id' del último lugar visitado (y el tipo de estar repetida). En su lugar, debería ser $max_{tr}(TI_{tr,K})$. **Se ha penalizado que se diga que el max debería ser un min, ya que eso implicaría que buscamos minimizar el camino más rápido y haría que tener un transporte sin visitas te alcanza optimalidad máxima.**
- (0.2) Puntos extra si comentáis las restricciones que proponéis, como su cardinalidad.

Puntuación máxima total: 2.6. Al haber tantos errores pequeños, hemos preferido dar margen para asegurarse que dejarse un par no descuenta demasiado.

También se han valorado con hasta 0.6 puntos extra representaciones alternativas que demostrasen comprensión de la técnica. A continuación presentamos una posible alternativa: La representación de este problema tiene la dificultad de que expandir las variables de ruta por camión hace crecer el espacio de búsqueda, cuando muchas de ellas tomarán por valor vacío y por ende no añaden nada. Las variables de visita se pueden cambiar por un conjunto de variables $NEXT_s$ y $FIRST_{tr}$, que indican a dónde se va a continuación de un sitio s (dominios $R \cup C \cup$) y a qué sitio va primero cada transporte. Esta representación sigue teniendo el problema de requerir un número muy alto de restricciones para comprobar la autonomía y el tiempo.

Comenta cada una de las soluciones que se proponen, analizando si la técnica escogida es adecuada para este problema, si cada uno de los elementos de la solución son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.